

SIAM

Algoritmos Genéticos

Prof.

Fernando S. de Aguiar Neto

1 Algoritmos Evolutivos

2 Estratégias

- Genótipo
- Mistura e Mutação
- Seleção
- Função de Aptidão - Fitness
- Exercícios e Discussões

Algoritmos Evolutivos

1 Algoritmos Evolutivos

2 Estratégias

Bio-Inspirado

- É uma estratégia de **otimização** inspirada na Teoria da Evolução
- Uma genética que traz uma característica que ajude na sobrevivência será repassada à diante (i.e. sobrevivência do mais apto)
- É um algoritmo de busca local, baseado em heurística; similar à subida de encosta, mas permitirá algumas perturbações para fugir de mínimos locais

Aplicações

- Tem grande aplicação em cenários onde o espaço de busca é combinatorial
- É genérico o suficiente para ser aplicado em vários problemas diferentes

Fluxo geral

- 1 Ranking dos indivíduos, segundo um critério de **aptidão**
- 2 Avaliar o(s) critério(s) de parada
- 3 Seleção de indivíduos
- 4 Criação de nova geração (por meio de mistura e mutação)
- 5 Repetir passos (cada ciclo é chamado de época)

Estratégias

1 Algoritmos Evolutivos

2 Estratégias

- Genótipo
- Mistura e Mutação
- Seleção
- Função de Aptidão - Fitness
- Exercícios e Discussões

Geral

- Cada etapa permitirá algumas variações
- Discutiremos alguns detalhes de parametrização e o impacto no funcionamento

População

- A cada época uma nova população de indivíduos (i.e. possíveis soluções será gerada
- Normalmente as populações possuem um tamanho fixo
- Isso garante que sempre haverão soluções a serem exploradas e que a memória ocupada permanecerá estável
- geralmente a primeira geração é gerada aleatoriamente

Critérios de parada

Os critérios de parada mais comuns são comuns a outros algoritmos:

- Quantidade de épocas
- Variação no ganho entre épocas
- Meta mínima de score (i.e. valor na função objetivo)
- Uma eventual combinação (e.g. o primeiro a ser atingido) desses critérios

Épocas versus População

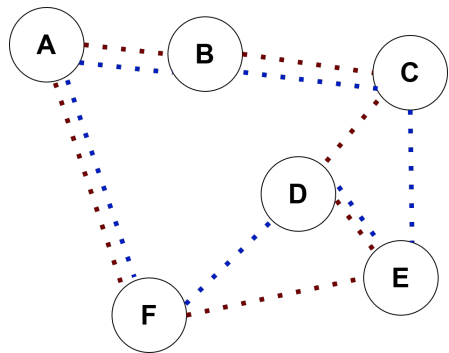
- Em geral a quantidade de soluções abordada será igual à quantidade de épocas multiplicada pela população
- Uma grande população permitirá uma maior ‘variedade genética’, evitando mínimos locais; a troco de um maior custo de espaço e tempo por época
- Uma quantidade maior de épocas permitirá que o algoritmo refine as soluções encontradas, melhorando a solução final

Genótipo

Descrevendo os indivíduos

- Cada indivíduo na nossa população será representado por um conjunto de genes
- Esses genes devem codificar a solução que desejamos encontrar
- Para ilustrar, vamos ver o problema do caixeiro viajante

O Problema do Caxeiro Viajante¹

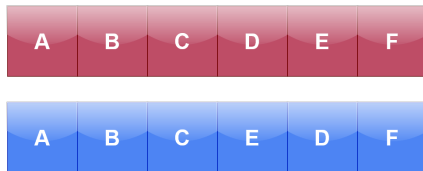
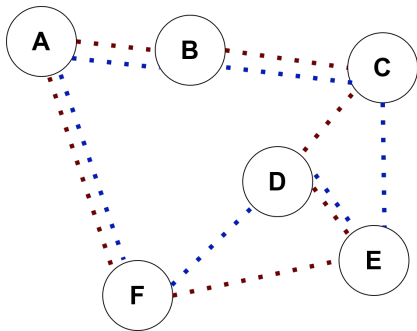


- O objetivo é visitar todas as cidades, voltando para a primeira (i.e. caminho cíclico)
- Minimizando a distância total percorrida
- O exemplo ao lado mostra dois caminhos possíveis em um exemplo simples onde todas as cidades são mutuamente alcançáveis

¹Apesar do exemplo ser um grafo totalmente conectado, o problema contempla pares de cidades inalcançáveis e distâncias assimétricas

Listas

- Geralmente as representações são vetores de uma dimensão
- No caso do caixeiro viajante, podemos colocar as cidades a serem visitadas, em ordem



Alternativas

- Qualquer alternativa é válida, desde que ela codifique a solução
- Discutiremos alguns cenários no final da aula
- Muito da literatura discute apenas vetores de uma dimensão, portanto é melhor adaptar coisas multidimensionais para uma dimensão
- Essa etapa é muito importante, e será base para toda a implementação

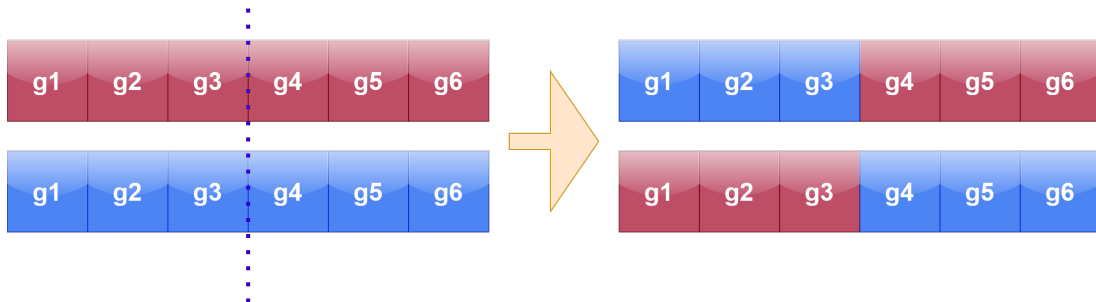
Mistura e Mutação

Gerando novos indivíduos

- Para gerar a nova geração esses genes precisam ser misturados
- Em geral são misturados os genes de dois indivíduos para gerar 1 ou mais indivíduos, dependendo dos critério para geração da nova população
- e.g. Podemos manter um dos ‘pais’ e gerar apenas 1 ‘filho’, mantendo a população constante
- e.g. Podemos gerar sempre 2 filhos e remover os pais da geração seguinte
- e.g. Podemos selecionar uma pequena parcela da população atual e gerar mais filhos

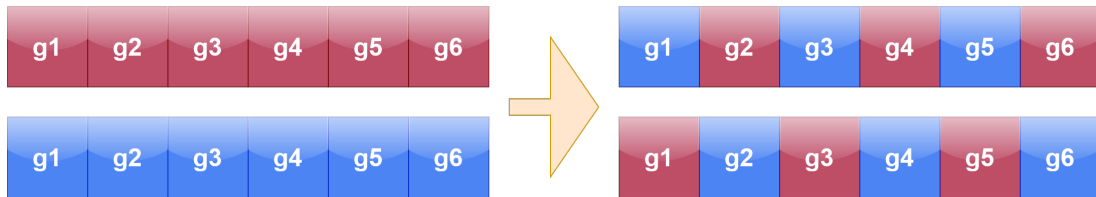
Crossover

- Podemos usar a primeira e segunda metades dos pais para gerar até 2 novos indivíduos



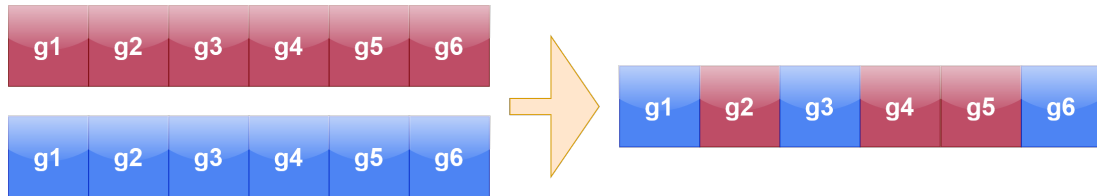
Crossover

■ Outra possibilidade



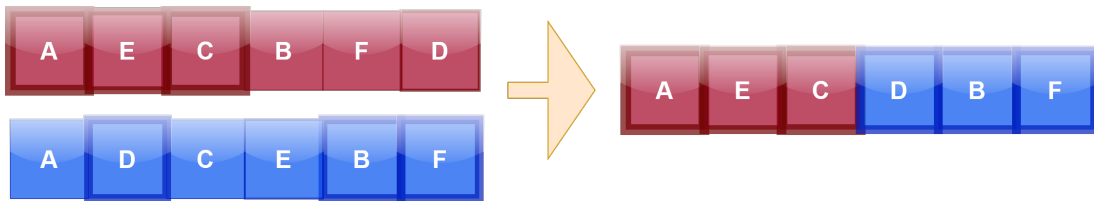
Uniform Crossover

- Ainda nessa premissa, podemos escolher aleatoriamente alguns genes de A e de B (com 50% de chance) para gerar o(s) novo(s) indivíduo(s)



Ordered Crossover

- Às vezes, como no caixeiro viajante, teremos que garantir que um mesmo valor não apareça em posições diferentes
- Podemos usar estratégias mais complicadas
- e.g. Mantemos a primeira metade de um dos pais e encontramos os valores faltantes e copiamos em ordem

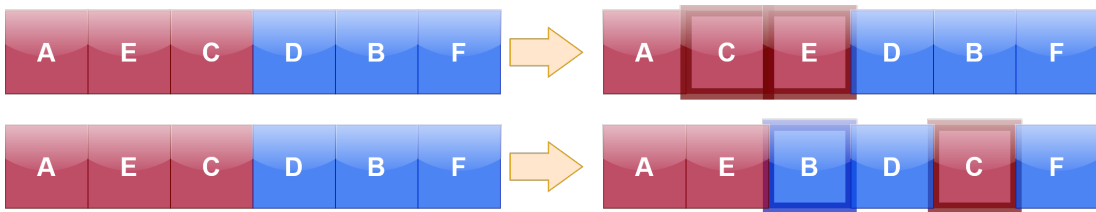


Mutações

- Para evitar mínimos locais podemos perturbar a genética dos novos indivíduos
- Geralmente existe uma chance associada à perturbação, e.g. 5% de chance de haver uma mutação durante a criação de um novo indivíduo
- Valores altos poderão perturbar de mais o espaço de busca, retardando o progresso realizado
- Por outro lado valores baixos podem demorar a gerar mudanças significativas para fugir de um máximo local

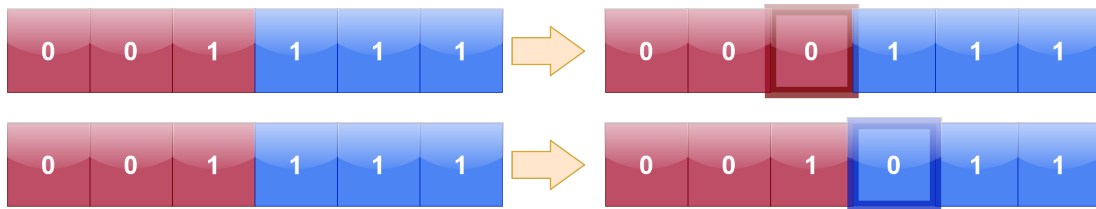
Swap

- A forma mais simples é a troca de posição de dois cromossomos



Flip

- Caso a representação de cada cromossomo seja binária, podemos apenas inverter o valor de 0 para 1 e vice-versa



Seleção

Seleção

- Os indivíduos mais aptos irão passar a genética à frente
- Mas temos algumas opções, tanto para a escolha dos ‘casais’, quanto daqueles que poderão ‘sobreviver’ fazer parte da geração seguinte

Sobreviventes

- Dependendo da forma escolhida para a mistura genética podemos decidir que todos ou uma parcela dos pais se manterão presentes na geração seguinte
- A ideia principal é manter soluções que foram efetivas até que elas sejam ultrapassadas pelas gerações seguintes
- Com isso em mente, veremos algumas formas de seleção e de ‘casamentos’

Top Ranking

- Os k indivíduos que tiverem a melhor função de aptidão (i.e. ranking) serão selecionados e passarão os genes à frente
- Útil para manter um controle sobre o tamanho da população
- Tende a manter soluções variadas

Top Performance

- Podemos definir um limiar para o score de aptidão² em relação à média e desvio padrão daquela geração
- Essa abordagem pode parecer promissora, mas pode reduzir a ‘variabilidade genética’, além de poder dificultar manter o tamanho da população controlado

²tipo uma nota de corte

Torneio

- Em casos onde as populações são muito grandes, podemos amostrar alguns indivíduos, fazendo ‘torneios’, o vencedor é selecionado
- Essa abordagem reduz a quantidade de cálculos de funções de aptidão, mas tenderá a demostrar mais para convergir
- Um eventual viés amostral pode ignorar soluções muito boas

Amostragem Estocástica

- Cada indivíduo pode ser escolhido aleatoriamente, em proporção à sua aptidão
- Essa abordagem permite manter qualquer solução independente da aptidão, priorizando as soluções melhores

Alpha

- O indivíduo que tiver o maior ranking irá se misturar com todos os outros selecionados (e.g. por top-ranking)
- Essa abordagem garante que um critério vencedor será perpetuado
- Mas pode tender a ficar preso em um mínimo local, já que toda a nova geração será muito similar

Função de Aptidão - Fitness

Aptidão

- A função de aptidão é o cerne da implementação, é a função que será otimizada pelo algoritmo
- Deverá ser algo que possa ser computado direta ou indiretamente usando a genética do indivíduo
- e.g. os genes codificam um conjunto de comandos que pode ser aplicado em um jogo, a aptidão é o score nesse jogo (indireto)
- e.g. os genes parametrizam uma função matemática que é calculada

Exemplo Cacheiro Viajante

- Por exemplo, podemos escolher a seguinte função para o cacheiro viajante
- $F(I) = [\sum_{i=0}^g d(I[i], I[i + 1])] + d(I[g + 1], I[0])$
- i.e. a soma das distâncias entre cidades adjacentes no caminho
- Caso a descrição dos genes permita, nosso algoritmo poderia não encontrar ciclos, ou fazer um ciclo de apenas 3 cidades
- Eventuais restrições devem estar na função objetivo ou na genética dos indivíduos

Aptidão - Detalhes

- Ainda que seja central, é mais fácil de ser adaptada ao decorrer de experimentos
- O algoritmo irá aproveitar qualquer brecha para otimizar essa função, então ela deve ser bem formulada

Exerícios e Discussões

Problemas de Satisfação de Restrições

- Vamos ver alguns problemas clássicos de satisfação de restrições que podem ser solucionados com algoritmos genéticos

Problema da Mochila

- Considerem uma mochila que aguente um peso limitado
- Um ladrão precisa colocar itens na mochila que maximizem o valor obtido, sem ultrapassar o limite de peso da mochila

n-Rainhas

- Queremos posicionar n -rainhas em um tabuleiro de xadrez de tamanho $n \times n$ ($n > 3$)
- As rainhas não devem se atacar

Alocação de Horários

- Queremos alocar professores à salas de aula e horários
- Uma sala deve ter apenas uma aula num determinado horário
- Um professor só pode dar aula para uma turma de cada vez
- Uma turma deve ter uma matéria de cada vez
- Idealmente as aulas de um professor devem se manter no mesmo campus ao longo de um dia (soft constraint)